AS6 工具链流水线优化设计

1. 结论

`AS6工具链流水线改造清单.md` 的主方向基本正确：它识别了 `ASCTG` 不是通用主链默认阶段、组件级 harness 场景中 `ASCTG` 应在 `MSCIGT` 前、`ASCTG` 产出的 harness project 应成为后续输入，以及 `AWAITING_CODE` 不应在仅执行 `ASCTG` 后出现。

但该清单仍需要优化：

- 依据应以 `ECOA_Exploitation_Guide.pdf` 的角色活动和教程 UC4-UC6 为主，而不是只抽象成一条工具顺序。
- “应用级集成模式”不能简单设计成 `EXVT -> CSMGVT -> LDP`。`LDP` 明确需要完整 ECOA 模型和所有模块源码或二进制交付；源码就绪不是该模式自动生成的结果，而是该模式的进入前提。
- Code Server 编辑代码的位置应作为明确闸口放在 `MSCIGT` 成功之后、`CSMGVT` / `LDP` 之前，并且继续执行时必须复用同一个 workspace，不能重新导出覆盖用户代码。
- 现有两模式划分还偏粗，建议至少区分“直接组件开发”和“harness 组件测试开发”，否则 `ASCTG` 会被误认为组件开发必经阶段。

优化后的目标是：把工具链从“所有工具串成一条自动流水线”调整为“按用户目标分流、带人工代码编辑闸口的状态机”。

2. PDF 依据摘要

以下依据来自 `ECOA_Exploitation_Guide.pdf`：

- 7.2.2 LDP：LDP 需要完整 ECOA 模型和已存在的模块源码，然后生成可执行 ECOA middleware；可用于开发、测试或演示。
- 7.2.3 MSCIGT：MSCIGT 生成 C/C++ 头文件、模块 skeleton、container 源码、部分模块级 harness 和构建文件，用于加速模块开发和测试。
- 7.2.4 ASCTG：ASCTG 从 ECOA 模型和待测组件集合生成新的 ECOA model，加入 harness component 并把待测子系统外部接口接到 harness。
- 7.2.5 CSMGVT：CSMGVT 生成非实时执行所需的最小 stub，让用户在没有 ECOA middleware 的工作站环境中做功能验证，并可结合 debugger。
- 7.2.6 EXVT：EXVT 是模型一致性和 AS6 合规检查工具，可在完整或局部模型上执行。
- 9.3 ASC Supplier 活动表：开发组件使用 `MSCIGT`；组件级测试可使用 `ASCTG + MSCIGT + LDP + debug tools`；组件功能测试可使用 `CSMGVT + debug tools`。
- 10.2.4 UC4：ASC Supplier 先用 `ASCTG` 为自己负责的组件子集生成 SAR-HARNESS 这类适配后的 ECOA model。
- 10.2.5 UC5：再用 `MSCIGT` 基于该模型生成源码骨架；生成的 module entry-point body 是空的，用户需要手工填写。
- 10.2.6 UC6：测试步骤是 `ASCTG -> MSCIGT -> 手工填写 harness/module 代码 -> CSMGVT`，或同样先完成手工代码后再用 `LDP` 在 ECOA environment 中测试。

因此，工具链的关键约束是：

1. `ASCTG` 是 harness 测试模型分支入口，不是所有场景的默认入口。
2. `MSCIGT` 是源码骨架生成点。
3. Code Server 应承接 `MSCIGT` 之后的人工代码补全。
4. `CSMGVT` 和 `LDP` 属于代码补全之后的验证 / 执行阶段。

3. 对现有改造清单的评审

3.1 正确项

现有清单中以下判断应保留：

- `ASCTG` 与 `MSCIGT` 在 harness 场景中的顺序应为 `ASCTG -> MSCIGT`。
- `ASCTG` 生成的是新的 harness ECOA model，后续工具必须切换到该 harness project。
- `ASCTG` 不应在应用级集成中默认启用。
- `AWAITING_CODE` 应绑定 `MSCIGT` 成功完成，而不是绑定任意建模阶段。
- 用户显式选择 `ASCTG` 但没有组件选择或 config 文件时，应失败并提示，而不是静默跳过。
- 任务历史应记录 `workflowMode`、`baseProjectFile`、`activeProjectFile`、`harnessProjectFile`。

3.2 需要修正或补强的项

现有清单中以下设计需要优化：

- “组件级开发 / 测试模式”把普通组件开发和 harness 测试开发混在一起。PDF 中 `MSCIGT` 可直接用于组件开发，`ASCTG` 只在需要为待测组件创建 harness 时使用。
- “应用级集成模式”默认 `EXVT -> 可选 CSMGVT -> LDP` 不完整。该流程必须先确认源码或二进制交付已经就绪，否则 `LDP` 无法满足输入前提。该确认不能只是一个勾选项，还需要来自上游任务、源码包导入、二进制交付或显式 Code Server 准备步骤的证据。
- Code Server 只写成一个流程步骤还不够。它应成为后端状态机中的显式暂停点：生成骨架后暂停，用户编辑后以 `continuing=true` 继续。
- 继续执行不能重新导出 EDT XML。重新导出可能覆盖 Code Server 中刚修改的业务代码，应使用 `skipExport=true` 并复用已持久化的 workspace。
- harness project 解析规则不能只取“非 base 的第一个 project 文件”。如果目录中存在多个派生 project，应优先使用 `ASCTG` 返回的项目文件，其次按 `*-harness.project.xml` 精确匹配。
- 历史记录除 project 文件外，还应记录源码状态，例如 `GENERATED_SKELETON`、`USER_EDIT_REQUIRED`、`USER_EDITED`、`SOURCE_READY`，否则 UI 很难判断是否可以继续执行 `CSMGVT` / `LDP`。

4. 优化后的工作模式

4.1 模式 A：组件直接开发模式

适用场景：

- 用户已经拥有完整组件上下文，不需要为缺失的相邻组件生成 harness。
- 目标是基于现有 ECOA model 生成模块 skeleton，然后补业务代码。

目标流程：

```
EDT/导入模型
  -> EXVT
  -> MSCIGT
  -> CODE_EDIT_REQUIRED
  -> Code Server 补齐 module entry-points、container mockup、unit test bodies
  -> 可选 CSMGVT
  -> 可选 LDP
```

默认阶段：

- 初始执行：`EXVT`，`MSCIGT`
- 继续执行：按用户选择运行 `CSMGVT` 和 / 或 `LDP`

状态要求：

- `MSCIGT` 成功后进入 `CODE_EDIT_REQUIRED`。
- 继续执行必须复用上次的 `activeProjectFile` 和 workspace。
- 如果用户只想生成 skeleton，可在 `CODE_EDIT_REQUIRED` 结束流程，不强制继续测试。

4.2 模式 B：组件 harness 测试开发模式

适用场景：

- ASC Supplier 只负责部分组件。
- 需要用 harness 替代不属于自己的相邻组件。
- 目标是生成可测试的派生 ECOA model，并补齐 harness / 被测组件代码。

目标流程：

```
EDT/导入模型
  -> EXVT
  -> ASCTG
  -> activeProjectFile = harnessProjectFile
  -> MSCIGT
  -> CODE_EDIT_REQUIRED
  -> Code Server 补齐被测组件代码、HARNESS module entry-points、测试输入/期望结果
  -> CSMGVT 非实时功能测试
  -> 可选 LDP ECOA environment 测试
```

默认阶段：

- 初始执行：`EXVT`，`ASCTG`，`MSCIGT`
- 继续执行：默认建议 `CSMGVT`；`LDP` 作为可选高级验证阶段

关键约束：

- `ASCTG` 必须有 selected component instances 或明确 config 文件。
- `ASCTG` 成功后必须切换 `activeProjectFile` 到 `*-harness.project.xml`。
- `MSCIGT` 必须基于 harness project 运行。
- Code Server 的打开目录应指向包含 harness project 和生成源码的 workspace，而不是孤立 output 目录。
- `CSMGVT` 和 `LDP` 都必须使用用户编辑后的同一份 harness workspace。

4.3 模式 C：应用级集成 / 系统验证模式

适用场景：

- 组件模型、部署模型、源码或二进制交付已经完成。
- 用户目标是系统级非实时功能验证、轻量平台运行、技术集成或演示。

源码就绪来源：

- 上游“组件直接开发模式”或“组件 harness 测试开发模式”已经完成，并将任务标记为 `SOURCE_READY`。
- 用户导入了已有源码目录或源码包，并且目录结构与 ECOA project 中的 component/module 引用匹配。
- 用户导入了组件二进制交付及对应 `bin-desc.xml`，走二进制集成路径。
- 用户在集成任务中选择“准备 / 编辑现有源码”，系统打开 Code Server，用户补齐后显式标记 `SOURCE_READY`。

如果以上条件都不满足，不能直接进入 `LDP`。此时产品应引导用户选择：

- 切换到“组件直接开发模式”生成 skeleton 并补代码。
- 切换到“组件 harness 测试开发模式”生成 harness、skeleton 并补代码。
- 导入已有源码 / 二进制交付。
- 打开 Code Server 在当前集成 workspace 中准备源码，然后再继续集成。

目标流程：

```
EDT/导入模型
  -> EXVT
  -> SOURCE_READY_CHECK
  -> SOURCE_PREP_REQUIRED? （无源码 / 二进制时）
  -> Code Server 或源码 / 二进制导入
  -> 可选 CSMGVT
  -> LDP
  -> 编译 / 执行 / 日志查看
```

默认阶段：

- 初始执行：`EXVT`，`LDP`
- 可选阶段：`CSMGVT`
- 默认不包含：`ASCTG`，`MSCIGT`

关键约束：

- 进入 `LDP` 前必须确认源码或二进制交付已就绪。
- 如果源码未就绪，UI 不应让用户继续跑 `LDP`，而应引导到“组件直接开发模式”、“组件 harness 测试开发模式”、源码 / 二进制导入，或集成 workspace 的 Code Server 准备步骤。
- 集成模式下 Code Server 不是 `MSCIGT` 后的自动暂停点；只有用户显式选择“准备 / 编辑现有源码”或编译失败需要修复时才打开。
- `ASCTG` 只在“交付组件验收”这种特殊测试分支中启用，不作为集成默认阶段。

5. Code Server 的正确位置

Code Server 应被设计为“人工补代码闸口”，而不是普通工具阶段。

5.1 进入条件

满足以下条件时进入 `CODE_EDIT_REQUIRED`：

- 当前模式是“组件直接开发模式”或“组件 harness 测试开发模式”。
- `MSCIGT` 成功完成。
- 后续执行阶段尚未开始。
- 当前任务没有失败。

不应进入 Code Server 的情况：

- 仅执行 `EXVT`。
- 仅执行 `ASCTG`。
- 应用级集成模式中已有源码 / 二进制就绪证据，或用户显式进入“准备 / 编辑现有源码”步骤。
- `MSCIGT` 失败。

5.2 Code Server 打开位置

Code Server 应打开到当前任务的 `Steps` workspace，理由是：

- `ASCTG` 派生的 harness model 位于该 workspace。
- `MSCIGT` 生成的源码骨架也应在该 workspace 中被编辑。
- 继续执行 `CSMGVT` / `LDP` 时需要消费同一份已编辑源码。

推荐记录字段：

```
{
  "workflowMode": "HARNESS",
  "workflowStage": "CODE_EDIT_REQUIRED",
  "baseProjectFile": "base.project.xml",
  "activeProjectFile": "base-harness.project.xml",
  "harnessProjectFile": "base-harness.project.xml",
  "codeWorkspacePath": "/workspace/<projectId>/<taskId>/Steps",
  "sourceState": "GENERATED_SKELETON"
}
```

5.3 继续执行要求

用户完成编辑后点击“继续测试 / 继续生成”：

- 请求必须带 `continuing=true`。
- 请求必须带上次持久化的 `activeProjectFile`。
- 请求必须 `skipExport=true`，防止重新导出覆盖用户代码。
- 允许阶段只应是 `CSMGVT`、`LDP` 或它们的子集。
- 成功继续后将 `sourceState` 更新为 `SOURCE_READY` 或 `USER_EDITED`。

5.4 失败回路

如果 `CSMGVT` / `LDP` 编译或执行失败：

```
FAILED
  -> 用户查看日志
  -> 打开 Code Server 修复
  -> continuing=true 重新运行失败阶段或后续阶段
```

这比把 Code Server 固定成一次性步骤更符合真实开发流程。

6. 推荐状态机

```
NEW
  -> EXPORTING_XML
  -> RUNNING_EXVT
  -> RUNNING_ASCTG?        (仅 harness 模式)
  -> SWITCH_ACTIVE_PROJECT?  (仅 ASCTG 成功后)
  -> RUNNING_MSCIGT?       (开发模式)
  -> CODE_EDIT_REQUIRED?    (仅 MSCIGT 成功后)
  -> READY_TO_CONTINUE
  -> RUNNING_CSMGVT?
  -> RUNNING_LDP?
  -> COMPLETED
```

失败状态：

```
RUNNING_* -> FAILED
FAILED -> CODE_EDIT_REQUIRED? -> READY_TO_CONTINUE
```

状态语义：

- `CODE_EDIT_REQUIRED`：系统已生成需要人工补齐的源码骨架。
- `READY_TO_CONTINUE`：用户确认已完成代码补齐，可以继续测试或平台生成。
- `SOURCE_READY_CHECK`：集成模式下的前置确认，不等同于系统生成了 skeleton；它只能消费上游开发任务、源码 / 二进制导入、或显式 Code Server 准备步骤产生的源码就绪证据。
- `SOURCE_PREP_REQUIRED`：集成模式缺少源码 / 二进制证据，必须先切换开发模式、导入交付物，或打开 Code Server 准备源码。

如果继续沿用现有状态名，可以把 `AWAITING_CODE` 保留为兼容别名，但 UI 文案应改为“等待补齐生成的源码骨架”，避免误导。

7. 阶段选择规则

7.1 阶段排序

建议统一使用以下 canonical order：

```
EXVT -> ASCTG -> MSCIGT -> CODE_EDIT_REQUIRED -> CSMGVT -> LDP
```

说明：

- `CODE_EDIT_REQUIRED` 是状态，不是外部工具。
- 非 harness 模式会跳过 `ASCTG`。
- 集成模式会跳过 `MSCIGT` 和 `CODE_EDIT_REQUIRED`，前提是源码已就绪。

7.2 阶段合法性

组件直接开发模式：

- 初始允许：`EXVT`，`MSCIGT`
- 继续允许：`CSMGVT`，`LDP`

组件 harness 测试开发模式：

- 初始允许：`EXVT`，`ASCTG`，`MSCIGT`
- 继续允许：`CSMGVT`，`LDP`

应用级集成模式：

- 初始允许：`EXVT`，`CSMGVT`，`LDP`
- 不允许：`ASCTG`，`MSCIGT`
- 如果用户需要 `MSCIGT`，说明当前目标不是集成模式，应切换到开发模式。
- 如果用户没有 `SOURCE_READY` / `BINARY_READY` 证据，`LDP` 不可执行，只能执行 `EXVT` 后进入 `SOURCE_PREP_REQUIRED`。

7.3 默认勾选

推荐 UI 默认值：

模式	初始默认	继续默认	说明
组件直接开发	`EXVT`，`MSCIGT`	无强制默认，推荐用户选择 `CSMGVT`	只生成骨架和进入 Code Server
组件 harness 测试开发	`EXVT`，`ASCTG`，`MSCIGT`	`CSMGVT`，`LDP` 可选	最贴近 UC4-UC6
应用级集成 / 系统验证	`EXVT`，`LDP`	不适用	需先确认源码或二进制就绪，`CSMGVT` 可选；缺少证据时只跑 `EXVT` 并进入准备步骤

8. 数据模型改造

建议任务记录新增或标准化以下字段：

```
{
  "workflowMode": "DIRECT_DEV | HARNESS_DEV | INTEGRATION",
  "workflowStage": "INITIAL | CODE_EDIT_REQUIRED | CONTINUING | COMPLETED",
  "baseProjectFile": "xxx.project.xml",
  "activeProjectFile": "xxx.project.xml",
  "harnessProjectFile": "xxx-harness.project.xml",
  "codeWorkspacePath": "/workspace/<projectId>/<taskId>/Steps",
  "sourceState": "UNKNOWN | GENERATED_SKELETON | USER_EDIT_REQUIRED | USER_EDITED | SOURCE_READY | BINARY_READY",
  "sourceReadinessEvidence": "UPSTREAM_TASK | SOURCE_IMPORT | BINARY_IMPORT | CODE_SERVER_PREP | MANUAL_CONFIRMATION",
  "skipExportOnContinue": true
}
```

字段语义：

- `baseProjectFile`：EDT 导出的原始 project。
- `activeProjectFile`：当前阶段实际消费的 project。`ASCTG` 成功后应变为 harness project。
- `harnessProjectFile`：ASCTG 产物，只在 harness 模式存在。
- `codeWorkspacePath`：Code Server 打开的目录，也是继续执行的输入根目录。
- `sourceState`：用于防止未补代码就运行 `CSMGVT` / `LDP`。
- `sourceReadinessEvidence`：记录源码 / 二进制就绪从哪里来。`MANUAL_CONFIRMATION` 只能作为过渡兼容方案，最终应尽量由导入动作或上游任务状态自动产生。

9. 分层改造清单

9.1 前端

- 增加工作模式选择：组件直接开发、组件 harness 测试开发、应用级集成 / 系统验证。
- 根据模式设置默认阶段和可选阶段。
- 在 `CODE_EDIT_REQUIRED` 状态显示“打开 Code Server”和“继续执行”两个动作。
- “继续执行”只展示 `CSMGVT` / `LDP`，不再展示 `EXVT` / `ASCTG` / `MSCIGT`。
- 集成模式进入 `LDP` 前要求选择源码就绪来源：使用上游任务输出、导入源码、导入二进制、打开 Code Server 准备源码，或过渡期手工确认。
- 缺少源码 / 二进制证据时，集成模式只允许先运行 `EXVT`，并在通过后进入 `SOURCE_PREP_REQUIRED`。
- 历史记录展示 `workflowMode`、`activeProjectFile`、`sourceState`、`codeWorkspacePath`。

9.2 Java 后端

- 请求 DTO 增加 `workflowMode`、`continuing`、`baseProjectFile`、`activeProjectFile`、`harnessProjectFile`、`sourceState`、`sourceReadinessEvidence`。
- 任务状态模型增加 `CODE_EDIT_REQUIRED` 或保留 `AWAITING_CODE` 兼容映射。
- 继续执行时强制带上 persisted `activeProjectFile`，并设置 `skipExport=true`。
- 任务回调持久化 active project 和 Code Server workspace。
- 阶段校验前置到后端，避免前端绕过模式限制。
- 对集成模式增加 source ready gate：没有 `SOURCE_READY` 或 `BINARY_READY` 时，不向 Python 编排层提交 `LDP` 阶段。

9.3 Python 编排层

- 将 workflow mode 从两个扩展为三个：`DIRECT_DEV`、`HARNESS_DEV`、`INTEGRATION`。
- 移除无 `workflowMode` 时的旧全链默认，或至少记录弃用警告并映射到明确模式。
- `ASCTG` 成功后优先读取工具返回的 harness project；没有返回时按 `*-harness.project.xml` 精确定位。
- `MSCIGT` 成功后返回 `CODE_EDIT_REQUIRED`，并带 `codeWorkspacePath`。
- 继续请求只允许 `CSMGVT` / `LDP`，并使用 persisted `activeProjectFile`。
- `CSMGVT` / `LDP` 运行前检查 `sourceState` 和 `sourceReadinessEvidence`，避免空 skeleton 或无源码 workspace 直接进入验证。
- 集成模式缺少源码 / 二进制证据时返回 `SOURCE_PREP_REQUIRED`，不要尝试运行 `LDP`。
- 所有工具日志输出当前使用的 project file，方便排查 harness 切换问题。

9.4 测试

至少增加以下测试：

- 组件直接开发：`EXVT -> MSCIGT -> CODE_EDIT_REQUIRED`。
- harness 开发：`EXVT -> ASCTG -> activeProjectFile 切换 -> MSCIGT -> CODE_EDIT_REQUIRED`。
- 仅执行 `ASCTG` 不进入 Code Server。
- 继续执行不会重新导出 EDT XML。
- 继续执行使用 persisted `activeProjectFile`。
- 集成模式缺少 source ready 确认时不能运行 `LDP`。
- 集成模式可以通过上游任务、源码导入、二进制导入或 Code Server 准备步骤进入 `SOURCE_READY` / `BINARY_READY`。
- 多个 project 文件存在时，harness project 解析不误选。
- `ASCTG` 缺 selected components/config 时失败。

10. 推荐落地顺序

P0：先修正流程语义

- 明确三种 workflow mode。
- 将 Code Server 改为 `MSCIGT` 后的暂停闸口。
- 强制继续执行复用 workspace，不重新导出。
- 修正 harness project 解析规则。
- `CSMGVT` / `LDP` 前增加 source ready 校验。

P1：完善前端和历史记录

- 前端按模式展示默认阶段和继续阶段。
- 历史记录展示 active project、source state、Code Server workspace。
- 编译 / 执行失败后提供“打开 Code Server 修复并重试”的回路。

P2：增强验收和文档

- 补充 UC4-UC6 对应的联调样例。
- 更新用户文档：说明什么时候使用 `ASCTG`，什么时候直接用 `MSCIGT`，什么时候进入集成模式。
- 补充排障说明：harness project 未切换、代码被重新导出覆盖、LDP 缺源码等问题。

11. 优化后的验收标准

改造完成后应满足：

1. harness 模式初始链路为 `EXVT -> ASCTG -> MSCIGT -> CODE_EDIT_REQUIRED`。
2. 直接开发模式初始链路为 `EXVT -> MSCIGT -> CODE_EDIT_REQUIRED`。
3. 集成模式默认不包含 `ASCTG` 和 `MSCIGT`。
4. Code Server 只在 `MSCIGT` 成功后自动出现。
5. 用户在 Code Server 中修改的源码不会被继续执行时重新导出覆盖。
6. `CSMGVT` / `LDP` 使用的是用户编辑后的 active project workspace。
7. `ASCTG` 成功后，后续 `MSCIGT` / `CSMGVT` / `LDP` 明确使用 harness project。
8. 仅 `ASCTG` 成功不显示“骨架代码已就绪”。
9. 集成模式运行 `LDP` 前必须能证明源码或二进制已就绪。
10. 前端、Java 后端、Python 编排层对模式、阶段、状态字段保持一致。
11. 集成模式不负责自动生成源码；如果没有上游源码、导入源码、导入二进制或 Code Server 准备结果，系统必须进入 `SOURCE_PREP_REQUIRED`，不能直接运行 `LDP`。